

Portable Fused GPU Kernels for Sensitive Protein Homology Search: A Triton Reformulation of the MMseqs2 Gapless Filter and Smith–Waterman–Gotoh Alignment

Document type: Research paper plan (pre-registration / experimental blueprint) **Target venue:** *Bioinformatics* (Oxford) — Original Paper; secondary option *NAR Genomics and Bioinformatics* **Status:** Plan only. The *Results* section is intentionally omitted; numerical claims appear as falsifiable hypotheses and success criteria to be measured on the GPU cluster using the companion runbook [02-experimental-protocol.md](#).

How to read this document. This is structured exactly like the target manuscript (Abstract → Introduction → Background → Objectives → Methods → Experimental Design → Expected Outcomes → Limitations → Reproducibility → References), but every place where a real paper would report a number, this plan states the *hypothesis* and the *measurement procedure* instead. The intent is that, after the experiments are run on the cluster, this file can be edited in place — replacing each "(to be measured, H#)" with the observed value — and become the manuscript draft.

Abstract

Protein sequence databases are growing faster than the compute available to search them, and sensitive homology search is now the dominant cost in both functional annotation and deep-learning structure prediction. The recently released GPU-accelerated MMseqs2 (MMseqs2-GPU) demonstrated that the sensitive *gapless filter* and *Smith–Waterman–Gotoh* (SWG) alignment stages can be moved onto GPUs for large speedups without sacrificing sensitivity. However, MMseqs2-GPU is implemented in hand-written CUDA that depends on NVIDIA-specific features (Hopper **DPX** instructions, **half2** packing, CUDASW++4.0), binding the fastest sensitive search tool to a single hardware vendor and imposing a high maintenance cost across hardware generations.

We propose to reimplement both GPU dynamic-programming stages as **fused kernels written in OpenAI Triton**, a Python-based compiler that emits optimized machine code for both NVIDIA (PTX/CUDA) and AMD (AMDGCN/ROCm) backends from a *single source*. Our central thesis is that a profiling-driven, fusion-oriented Triton implementation can (i) match CUDA-class throughput on NVIDIA while preserving **bit-exact filter scores** (and therefore identical sensitivity), (ii) deliver the **first sensitive GPU homology-search filter that runs on AMD GPUs**, which the CUDA implementation structurally cannot, and (iii) reduce kernel launches and global-memory round-trips by fusing PSSM lookup, low-complexity masking, the gapless recurrence, and candidate selection into one kernel. We will validate the approach on the established MMseqs2 SCOP/UniProt sensitivity benchmark (6,370 queries against 30.4 M reference sequences), synthetic and real-sequence TCUPS throughput benchmarks, database-scaling and multi-GPU streaming experiments, and two end-to-end downstream applications — ColabFold MSA generation on CASP14 free-modeling targets and Foldseek structural search — across an NVIDIA (A100/H100/L40S) and AMD (MI300X) hardware matrix. This plan specifies the algorithms, kernel designs, fusion strategy, correctness methodology, baselines, datasets, metrics, ablations, and statistical protocol in full; only the measured results are deferred.

Keywords: protein homology search; MMseqs2; GPU acceleration; Triton; kernel fusion; cross-vendor portability; gapless filter; Smith–Waterman–Gotoh; PSSM; structure prediction.

1. Introduction and Motivation

1.1 The widening sequence–compute gap

Identifying evolutionarily related sequences (homologs) in large reference databases underpins protein function inference, comparative genomics, and — increasingly — deep-learning structure prediction, where the quality of the input multiple sequence alignment (MSA) directly determines the accuracy of methods such as AlphaFold2 and ColabFold. Sequencing throughput has historically outpaced the growth of single-core compute, so for metagenomic-scale corpora the protein-search step dominates total cost. MMseqs2 (Steinegger & Söding, 2017) closed much of this gap on CPUs through a three-stage pipeline — a similar- k -mer double-match prefilter, a vectorized ungapped alignment, and a gapped Smith–Waterman alignment — reaching BLAST-level sensitivity at orders-of-magnitude higher speed.

1.2 GPUs closed the throughput gap — but only on NVIDIA

MMseqs2-GPU (Kallenborn et al., 2025) re-expressed the two most expensive stages as GPU dynamic-programming (DP) kernels: a **gapless filter** operating on position-specific scoring matrices (PSSMs) and a **Smith–Waterman–Gotoh** gapped aligner adapted from CUDASW++4.0. Reported gains are large — up to 13.5 trillion cell updates per second (TCUPS) on a single L40S (~58% of that GPU's theoretical peak), single-query searches ~6.4× faster than BLAST and ~177× faster than JackHMMER, and a 31.8× end-to-end speedup of the ColabFold/AlphaFold2 MSA pipeline — while preserving sensitivity (ROC1 ≈ 0.40/0.612/0.669 at 1/2/3 search iterations, matching the CPU implementation). Memory demand drops from ~7 bytes to ~1 byte per residue, and database streaming plus multi-GPU sharding extend the method beyond GPU-memory limits.

This is a decisive result for NVIDIA hardware. It is also, by construction, **not portable**. The implementation relies on:

- **Hopper DPX / packed half2 / s16x2 arithmetic** to reach one cycle per cell update (the basis of the 23.2 TCUPS theoretical peak on L40S);
- **warp-shuffle** intrinsics and **shared-memory bank-conflict** layouts specific to the CUDA execution model;
- **CUDASW++4.0** for the gapped stage.

None of these run on AMD GPUs. As AMD's CDNA-class accelerators (e.g., MI300X: 192 GB HBM3, ~5.3 TB/s bandwidth) become widely available in HPC and cloud settings — and given that the gapless filter is a *memory-bandwidth-bound* problem where HBM3 capacity and bandwidth are exactly the scarce resources — the inability to target them is a real and growing limitation. Vendor lock-in also raises the maintenance burden: each new GPU generation requires re-tuning hand-written CUDA.

1.3 Triton offers portability and fusion without CUDA expertise

OpenAI Triton (Tillet et al., 2019) is a block-based GPU programming language that compiles a single Python source to optimized code for both NVIDIA and AMD backends, automating memory coalescing, shared-

memory management, and intra-block scheduling. Two recent applications to scientific simulation establish the methodology we will follow:

- **TensorNet/TorchMD-NET acceleration** (Singh, 2025) used profiling-driven kernel fusion to combine 3–8 PyTorch operations into single Triton launches, reducing kernel launches by 75–88% and achieving 2.82–3.14× speedups on memory-bound message-passing and tensor-decomposition operations, while verifying **numerically identical** outputs.
- **Triton-accelerated state-space models for ICU monitoring** (Anonymous, 2025) fused irregular-sampling preprocessing and inference into one kernel for a 35.7× end-to-end latency reduction, and — critically for us — demonstrated that the *same Triton source* runs on an AMD MI300X with <8% latency overhead versus NVIDIA, with a rigorous 5-seed statistical protocol and full roofline/occupancy characterization.

These works show that (a) fusion of memory-bound kernels yields order-of-magnitude reductions in launches and memory traffic, and (b) Triton's cross-vendor portability is real and measurable. Neither has been applied to biological sequence search.

1.4 Our contribution (proposed)

We propose **MMseqs2-Triton**: fused Triton kernels for the gapless filter and SWG alignment, integrated into MMseqs2 and benchmarked against MMseqs2-GPU (CUDA). The contributions, stated as claims to be tested, are:

1. **A portable fused gapless-filter kernel** that folds PSSM lookup, on-the-fly low-complexity (tantan) soft-masking, the gapless DP recurrence, and candidate thresholding/compaction into a single Triton launch, compiled unchanged for NVIDIA and AMD.
2. **A portable SWG gapped-alignment kernel** using an anti-diagonal wavefront, with an honest fallback strategy where Triton cannot match hand-tuned CUDA.
3. **The first sensitive GPU homology-search filter that runs on AMD GPUs**, quantified against AMD CPU baselines (the only available AMD reference, since CUDA MMseqs2-GPU cannot execute there).
4. **A profiling-driven fusion analysis** reporting kernel-launch reduction, achieved HBM bandwidth vs. roofline, and occupancy on both vendors.
5. **A guarantee of sensitivity neutrality** via bit-exact filter scores, validated end-to-end on ColabFold (TM-score) and Foldseek (SCOPE sensitivity).

The unifying thesis: *the fastest sensitive homology search need not be the least portable.*

2. Background and Related Work

2.1 Heuristic sensitive search and the filter–align paradigm

Exact Smith–Waterman–Gotoh (Smith & Waterman, 1981; Gotoh, 1982) guarantees optimal gapped local alignment but is too slow at database scale. Practical tools therefore *filter then align*: BLAST/PSI-BLAST (Altschul et al., 1990, 1997) use seed-and-extend; DIAMOND (Buchfink et al., 2021) uses spaced-*k*-mer colinear comparison for cache locality; MMseqs2 (Steinegger & Söding, 2017) uses similar-*k*-mer double matches on a shared diagonal. Sensitive HMM tools — HMMER/JackHMMER (Eddy, 2011) and HHblits (Steinegger et al., 2019) — instead rank candidates with a *gapless* DP that finds the best substitution-only

alignment, trading speed for sensitivity. MMseqs2-GPU adopts this gapless-filter philosophy because, unlike word-based filtering, it does not trade sensitivity for speed — essential for structure-prediction MSAs.

2.2 The two GPU dynamic-programming stages we target

Gapless filter. For PSSM Q of length m (over alphabet Σ) and reference sequence $S = (s_1, \dots, s_n)$, the gapless score is computed by the recurrence

$$\begin{aligned} M[i, j] &= \max(M[i-1, j-1] + Q[i, s_j], \ 0 \), & 1 \leq i \leq m, \ 1 \leq j \leq n \\ M[i, 0] &= M[0, j] = 0 \\ \text{score} &= \max \text{ over all } (i, j) \text{ of } M[i, j] \end{aligned}$$

The only data dependency is on the **diagonal** neighbour $M[i-1, j-1]$. This is the algorithmic crux: every cell in a given row depends only on the previous row, so an entire row can be computed in parallel, with the diagonal coupling handled by a one-step shift between threads/lanes. MMseqs2-GPU exploits this with: PSSM in shared memory; reference residues in global memory (1 byte/residue); 16-bit packed arithmetic (**half2/s16x2**, Hopper **DPX**); thread groups of 4/8/16 with up to 128 cells per thread; grid-searched tile sizes (up to 2048 columns; longer queries tiled with the last column spilled to global memory); **warp shuffles** for the diagonal exchange; a **column-permutation** trick that packs diagonally-adjacent columns (0,16),(1,17),... into the same 32-bit word so vectorized lanes carry a clean one-step dependency; and a two-copy PSSM layout to remove shared-memory bank conflicts for group size 4. Reported efficiency is ~58% of the L40S theoretical peak.

Smith–Waterman–Gotoh (gapped). With affine gaps, each cell depends on its top, left, and diagonal neighbours (plus gap-state matrices E, F), precluding row-parallelism. MMseqs2-GPU uses an **anti-diagonal wavefront** (threads work on different rows along a minor diagonal), 32-bit accumulators to avoid overflow, thread groups of 4/8/16/32, and the profile transposed onto the x-axis, reusing CUDASW++4.0 machinery.

2.3 GPU programming, fusion, and portability

Operator fusion via **torch.compile**/TorchScript is automatic but lacks fine-grained control for domain-specific access patterns; hand-written CUDA (e.g., cuEquivariance, CUDASW++4.0) is maximally fast but vendor-locked and maintenance-heavy. Triton occupies the middle ground: block-based kernels with autotuning, compiled for both vendors. FlashAttention (Dao et al., 2022) established the IO-aware fusion philosophy — keep data in SRAM, minimize HBM round-trips — that we apply to the gapless recurrence. The TensorNet and ICU-SSM Triton works (Section 1.3) provide the concrete profiling → fuse → verify → benchmark template and the cross-vendor evidence base.

2.4 Gap analysis

Capability	MMseqs2-CPU	MMseqs2-GPU (CUDA)	MMseqs2-Triton (proposed)
Sensitive gapless filter	✓ (SIMD)	✓	✓ (bit-exact target)
Runs on NVIDIA GPUs	—	✓	✓ (single source)

Capability	MMseqs2-CPU	MMseqs2-GPU (CUDA)	MMseqs2-Triton (proposed)
Runs on AMD GPUs	—	✗ (CUDA/Hopper-specific)	✓ (novel)
Single source, multi-vendor	—	✗	✓ (novel)
Fused mask + DP + select	partial	partial (DP fused)	✓ (extended fusion)
Maintenance across HW gens	—	high (hand-CUDA)	low (autotuned Triton)
Sensitivity-for-speed trade-off	none (gapless)	none	none (preserved)

The white space is unambiguous: **portable, fused, sensitivity-preserving GPU search**. That is the paper.

3. Research Objectives and Hypotheses

We pre-register four falsifiable hypotheses. Each names the metric, the comparison, and the decision rule; measurement procedures are in [02-experimental-protocol.md](#).

- **H1 — Throughput parity on NVIDIA (with exact sensitivity).** The fused Triton gapless filter achieves $\geq 0.85\times$ the TCUPS of MMseqs2-GPU (CUDA) on the same NVIDIA GPU (L40S/H100/A100), at every sequence-length tile in 32–2048, while producing **bit-exact** gapless filter scores ($\max |\Delta\text{score}| = 0$) and therefore **identical ROC1 sensitivity** (target 0.40/0.612/0.669 at 1/2/3 iterations on the SCOP/UniProt benchmark). *Falsified if* TCUPS < 0.85× or any score differs or ROC1 deviates beyond the benchmark's reporting precision.
- **H2 — Cross-vendor portability (the headline claim).** The *identical* Triton source compiles and runs on AMD MI300X, yielding a working **sensitive gapless GPU filter on AMD** — a capability MMseqs2-GPU (CUDA) does not possess. On MI300X the Triton filter is $\geq 10\times$ faster than MMseqs2-CPU gapless on a matched-cost AMD EPYC CPU configuration, and within a documented factor of its own NVIDIA throughput. *Falsified if* the source fails to run on ROCm without algorithmic changes, or fails to beat the AMD CPU baseline.
- **H3 — Fusion reduces launches and memory traffic.** Folding PSSM lookup + tantan soft-masking + gapless DP + candidate thresholding/compaction into one kernel reduces prefilter kernel launches by $\geq 50\%$ versus an unfused Triton staging, eliminates the full-length score array's HBM round-trip, and raises achieved HBM bandwidth toward the roofline (target $\geq 50\%$ of peak on both vendors). *Falsified if* launch count or achieved bandwidth does not improve over the unfused baseline.
- **H4 — Downstream accuracy is unchanged.** Substituting MMseqs2-Triton for MMseqs2-GPU in ColabFold leaves CASP14 free-modeling TM-score statistically indistinguishable ($\approx 0.70 \pm 0.05$; paired test 95% CI includes 0), and Foldseek SCOPe family/superfamily/fold sensitivity unchanged within seed variance. *Falsified if* any downstream quality metric degrades beyond seed noise.

Primary objective: establish MMseqs2-Triton as a *portable, sensitivity-neutral* alternative to CUDA-only MMseqs2-GPU. **Secondary objective:** characterize *where* Triton wins and loses versus hand-tuned CUDA

(the "when CUDA wins" analysis), so practitioners can dispatch optimally.

4. Methods

This section is the core contribution and is written to maximal specificity. Stages are built and validated in the order presented.

4.1 Problem formalization and notation

Inputs: a query PSSM $Q \in \mathbb{R}^{m \times |\Sigma|}$ ($|\Sigma| = 20$ amino acids, optionally +X/ambiguity), constructed from a single query plus a substitution matrix or from an iterative sequence profile; and a reference set $\{S_r\}$ packed as 1 byte/residue with a length/offset index. Outputs: (filter) a gapless score per reference and a compacted list of candidates passing an inclusion threshold; (align) SWG scores and traceback for the surviving candidates. All kernels target the existing MMseqs2 on-disk database format so the work is a drop-in module behind `--prefilter-mode 1 --gpu 1`.

4.2 Profiling-driven bottleneck analysis (Phase 0)

Following the TensorNet/ICU methodology, we first profile a reference pipeline to localize memory-bound bottlenecks and set fusion priorities, **before** writing optimized kernels.

- **Tooling:** NVIDIA Nsight Compute / Nsight Systems on NVIDIA; `rocprow/omniperf` on AMD; PyTorch profiler for any PyTorch-staged reference implementation.
- **Baselines profiled:** (a) MMseqs2-GPU CUDA (NVIDIA) as the performance target; (b) a deliberately *unfused* Triton/PyTorch staging of the same math, to expose the launch-count and HBM-round-trip cost that fusion will remove.
- **Metrics captured per kernel:** time %, call count, achieved occupancy, registers/thread, shared-memory/block, achieved HBM bandwidth, L2 hit rate, warp/wavefront divergence, arithmetic intensity (FLOP/byte) → roofline placement.
- **Deliverable:** a bottleneck table (analogous to the TensorNet Table 1) ranking operations by time and tagging each as fusion-amenable, plus a roofline plot establishing that the gapless filter is bandwidth-bound (expected, given ~1–2 ops/cell and 1 byte/residue streaming).

4.3 Fused Triton gapless-filter kernel (Phase 1 — primary contribution)

Block decomposition. One Triton program instance computes the gapless alignment of one query tile against one reference sequence (or a length-bucketed group of references for load balance). The grid is `(num_reference_sequences, num_query_tiles)`; long queries (> tile width, up to 2048 columns as in the CUDA design) are tiled along m , spilling each tile's boundary column to a scratch buffer that seeds the next tile.

Inner loop (the recurrence). We iterate the *sequential* axis (PSSM rows $i = 1..m$) in a Python `for/tl.range` loop. At each row, a `BLOCK_J` span of reference positions is processed in parallel with `tl.arange`. The diagonal dependency $M[i-1, j-1]$ is realized as a **one-lane shift** of the previous row's register vector (intra-block shift via `tl.where/index` roll; the block-boundary carry is passed in a register/scratch). Per cell we perform: (1) gather the PSSM contribution $Q[i, s_j]$ (a lookup into the loaded PSSM row using the reference residue s_j as index), (2) add to the shifted diagonal value, (3) clamp at 0

(`tl.maximum(x, 0)`), (4) update a running max accumulator. The running max is reduced (`tl.max`) and written once per reference.

Memory strategy (IO-aware, FlashAttention-style).

- PSSM tile resident in SRAM for the lifetime of the block (reused across all reference positions) — the analogue of keeping K/V tiles on-chip.
- Reference residues streamed from HBM with **coalesced** `tl.load` (contiguous 1-byte residues; vectorized loads where alignment permits).
- No intermediate **M** matrix is materialized in HBM — only the running max lives in registers — eliminating the dominant memory traffic.

Vectorization and packing. We use packed low-precision arithmetic (**bf16** primary; **fp16** and **fp32** as ablation arms). We replicate the CUDA **diagonal-packing permutation** (pack columns that are diagonally adjacent into one vector so the packed dependency is a clean one-step carry), expressed as a compile-time block layout in Triton. On NVIDIA this should map onto packed-math units; on AMD, Triton lowers to the CDNA equivalent — the portability payoff is that *we do not hand-write either*.

Fusion targets (the H3 contribution). Into this single kernel we additionally fold:

1. **On-the-fly tantan soft-masking:** low-complexity reference residues are mapped to X (neutral/zero contribution) at load time, removing the separate DB-masking pass.
2. **Thresholding + candidate compaction:** instead of writing all N scores to HBM for a later sort, each block emits only references whose score clears the inclusion threshold, via an atomically-incremented output cursor (a compaction write). This removes a full-length HBM write+read and a separate selection kernel.

We will report the **launch-count reduction** and **HBM-traffic reduction** from each fusion increment (an ablation that attributes the speedup, mirroring TensorNet §4.5).

Autotuning. `@triton.autotune` over **BLOCK_J**, cells/thread, **num_warps**, **num_stages**, and group size, with separate best-config tables per (GPU, query-length bucket) — and, crucially, **re-autotuned per vendor**. We will report whether NVIDIA-optimal configs transfer to AMD (a portability-of-tuning result).

Pseudocode (fused gapless filter):

```
@triton.autotune(configs=[...], key=["m", "BLOCK_J", "dtype"])
@triton.jit
def gapless_filter_fused(Q_ptr, S_ptr, Slen_ptr, mask_ptr,
                        out_idx_ptr, out_score_ptr, cursor_ptr,
                        m, threshold, BLOCK_J: tl.constexpr, DT:
tl.constexpr):
    r = tl.program_id(0) # reference sequence id
    qt = tl.program_id(1) # query tile id
    Qtile = load_pssm_tile_to_sram(Q_ptr, qt) # resident in SRAM
    diag = tl.zeros([BLOCK_J], dtype=DT) # M[i-1, j-1] carry
    runmax = tl.zeros([BLOCK_J], dtype=DT)
    for i in range(0, m): # sequential over PSSM
        rows
            sj = load_residues_coalesced(S_ptr, r, i_window=BLOCK_J) # 1
        byte each
```

```

        sj    = apply_tantan_softmask(sj, mask_ptr)                #
fused mask -> X
        qv    = gather_pssm_row(Qtile, i, sj)                    #
Q[i, s_j]
        cur   = tl.maximum(shift_one(diag) + qv, 0)              #
gapless recurrence
        runmax = tl.maximum(runmax, cur)
        diag = cur
        score = tl.max(runmax)
        if score >= threshold:                                # fused thresholding +
compaction
        slot = tl.atomic_add(cursor_ptr, 1)
        tl.store(out_idx_ptr + slot, r)
        tl.store(out_score_ptr + slot, score)

```

(Indicative; final layout follows Phase-0 profiling and the diagonal-packing permutation.)

4.4 Fused Triton Smith–Waterman–Gotoh kernel (Phase 2 — secondary, with contingency)

The gapped stage runs only on the small candidate set surviving the filter, so it is less throughput-critical but needed for the end-to-end story.

Design. Anti-diagonal **wavefront** within a block: along each minor diagonal, cells are independent and computed in parallel; the kernel loops over diagonals sequentially. We maintain three score lanes (M, E, F) for affine gaps with 32-bit accumulators (overflow-safe), profile transposed onto the parallel axis, and a candidate per thread group (sizes 4/8/16/32 as ablation).

Honest risk and contingency. Wavefronts are inherently sequential along the diagonal and data-dependent — exactly the pattern where the TensorNet/ICU papers found Triton can lose to hand-tuned CUDA (atomic/reduction overheads, divergence). We therefore pre-commit to a **decision gate** (Phase-2 milestone): if the Triton SWG kernel cannot reach a preset fraction (e.g., 0.7×) of CUDASW++4.0 throughput after autotuning, we **retain the CUDA SWG kernel** and ship a **hybrid dispatcher** — portable Triton filter everywhere, CUDA gapped stage on NVIDIA, Triton gapped stage on AMD (where CUDA is unavailable, so even a slower portable kernel is strictly enabling). This preserves the H2 portability claim regardless of the SWG outcome and is reported transparently, not hidden.

4.5 Cross-stage fusion and launch-count accounting (Phase 3)

Beyond intra-kernel fusion (§4.3), we fuse the **producer → consumer handoff**: the filter's compacted candidate buffer feeds the SWG kernel directly (device-resident indices, no host round-trip), and where profitable we fuse short pre/post steps (e.g., score normalization, E-value setup) that currently launch separately. We will tabulate launches for the full prefilter → align path: unfused Triton vs. fused Triton vs. CUDA MMseqs2-GPU, reporting the percentage reduction (the H3 headline, analogous to TensorNet's 75–88%). We will be explicit that MMseqs2-GPU's CUDA DP is *already* a single fused kernel — our fusion delta is the *added* masking/selection/handoff folding plus portability, not a claim that CUDA was unfused.

4.6 Cross-vendor compilation and execution strategy (Phase 4)

- **Single source, two backends.** The same `.py` Triton kernels are compiled to PTX (NVIDIA, CUDA 12.x) and AMDGCN (AMD, ROCm 6.x). No `#ifdef`-style divergence is permitted in the algorithm; only

autotune config sets differ.

- **Vendor-neutral peak model.** We extend the MMseqs2-GPU theoretical-peak-performance (TPP) model, $TPP = (\#SMs \times \text{throughput_per_instruction} \times \text{clock}) / \text{cycles_per_cell_update}$, to AMD by substituting CDNA3 CU count, packed-math throughput, and clock, giving a per-vendor efficiency (% of peak) — the fair way to compare across architectures with different bandwidth/compute.
- **Scaling.** Reuse MMseqs2-GPU's database sharding across GPUs and asynchronous host → device streaming (overlap batch $i+1$ transfer with batch i compute) so databases exceeding GPU memory stream at a documented fraction of in-memory speed; verify the streaming path on both vendors (MI300X's 192 GB HBM3 should reduce streaming pressure).
- **Persistent GPU server.** Reuse the MMseqs2 GPU-server mode (persistent context + Linux shared memory) to amortize the ~300 ms context-init cost across the repeated `ungappedprefilter` invocations in ColabFold workflows.

4.7 Numerical-correctness methodology (gates every phase)

Correctness is a release gate, not an afterthought (mirroring TensorNet §4.4 and ICU's $<5 \times 10^{-7}$ verification):

- **Filter scores:** require **bit-exact** equality to the CPU/CUDA gapless score for every (query, reference) pair on a held-out correctness set (integer/exactly-representable accumulation path; `bf16` arm checked for any deviation and only shipped if scores remain identical or sensitivity is provably unaffected).
- **SWG:** require identical optimal scores (and consistent tracebacks up to documented tie-breaking) versus CUDASW++4.0.
- **End-to-end:** ROC1 on the SCOP/UniProt benchmark must match MMseqs2-GPU to reported precision; any kernel failing a gate is not deployed (the "deploy only kernels that pass" rule).

4.8 MMseqs2 integration and artifact

Kernels are integrated into the MMseqs2 module structure behind existing flags (`--prefilter-mode 1`, `--gpu 1`, `--gpu-server 1`) so downstream tools (ColabFold, Foldseek) consume them unmodified. The artifact is an open-source branch plus a standalone benchmark harness (CUDASW++-style) used for the kernel-level measurements.

5. Experimental Design

Full operational detail (commands, dataset URLs, sweep tables, results templates) is in `02-experimental-protocol.md`. This section defines *what* will be measured and *against what*.

5.1 Datasets and benchmarks (reusing established, reviewer-trusted setups)

1. **Sensitivity (SCOP/UniProt).** The MMseqs2 benchmark: 6,370 query sequences; reference = 3.4 M annotated UniProt + 27 M reversed sequences = **30,430,281** references; full-length queries retaining disordered/low-complexity/repeat regions. True positives = same SCOP family; false positives = reversed or different-fold. Metric: **ROC1** (AUC to first false positive). Run at 1/2/3 profile-search iterations.
2. **Throughput (TCUPS).** (a) *Synthetic*: uniform-length queries vs. 5 M uniform-length references for each tile length $l \in \{32, \dots, 2048\}$ (best-case utilization). (b) *Real*: the 6,370 queries vs. the 30.4 M database. Report TCUPS and % of per-vendor theoretical peak.

3. **Database scaling & streaming.** 1×/4×/16× replicated reference DB to cross the GPU-memory boundary; report in-memory vs. streamed TCUPS and the streamed/in-memory ratio.
4. **End-to-end structure prediction.** ColabFold MSA generation for **20 CASP14 free-modeling targets**; compare AlphaFold2 (JackHMMER+HHblits) vs. ColabFold-MMseqs2-CPU vs. ColabFold-MMseqs2-GPU(CUDA) vs. **ColabFold-MMseqs2-Triton**; metric: end-to-end wall time and **TM-score** (accuracy neutrality, H4).
5. **Structural search transfer.** Foldseek 3Di search, 6,370 structures sampled from AFDB50, with the Triton filter substituted; metric: speed and SCOPe family/superfamily/fold sensitivity (H4 transfer).

5.2 Baselines

- **Primary (NVIDIA):** MMseqs2-GPU (CUDA) — the SOTA target for H1/H3.
- **CPU references:** MMseqs2-CPU gapless (SIMD, `--prefilter-mode 1`) and MMseqs2-CPU *k*-mer (`-s 8.5`).
- **AMD:** MMseqs2-CPU on AMD EPYC (the *only* available AMD baseline, since CUDA MMseqs2-GPU cannot run there) — this is the comparison that substantiates H2.
- **Context (sensitivity/speed landscape):** BLAST, PSI-BLAST, DIAMOND (ultra-sensitive), JackHMMER — for positioning, parameterized to matched ROC1 where possible.

5.3 Hardware matrix

Vendor	GPUs	Host	Backend
NVIDIA	L40S (primary), H100, A100	2× AMD EPYC, ≥1 TB RAM	CUDA 12.x, Triton (PTX)
AMD	MI300X (192 GB HBM3)	AMD EPYC	ROCm 6.x, Triton (AMDGCN)

Energy/cost measured on a representative cloud-equivalent configuration (AWS EC2 instance classes matching each setup) for \$/query and J/query.

5.4 Metrics

- **Throughput:** TCUPS and % of theoretical peak (per-vendor TPP model).
- **Speed:** per-query latency; batch throughput (single, 10, 100, 6,370); tail latency (p50/p95/p99) for the single-query/server scenario.
- **Fusion:** kernel-launch count (and % reduction); HBM round-trips eliminated.
- **Memory system:** achieved HBM bandwidth (GB/s and % of peak), occupancy, registers/thread, L2 hit rate, warp/wavefront divergence (Nsight Compute / rocprof).
- **Sensitivity & accuracy:** ROC1; CASP14 TM-score; Foldseek SCOPe sensitivity.
- **Correctness:** max |Δscore| vs. CPU/CUDA (target 0 for filter).
- **Economics:** cloud \$/query; energy J/query.

5.5 Ablation grid (attribution of gains)

1. **Precision:** fp32 / **bf16** / fp16 — TCUPS and sensitivity; document fp16 NaN risk on extreme scores (per the ICU paper's fp16 finding) and recommend bf16 if score-neutral.
2. **Fusion level:** unfused → +mask → +threshold/compaction → +cross-stage handoff — isolates each fusion's contribution to launches/bandwidth/TCUPS.

3. **Tiling/scheduling:** `BLOCK_J`, cells/thread, thread-group size, `num_warps`, `num_stages` — autotune sweep; report best per (vendor, length bucket).
4. **Sequence-length scaling:** 32–2048 and long-query tiling > 2048.
5. **Scaling out:** 1/2/4/8 GPUs; in-memory vs. streamed DB.
6. **Cross-vendor:** every primary measurement repeated on NVIDIA and AMD; report config transferability.

5.6 Statistical protocol

Throughput/latency: median over ≥ 5 runs after ≥ 20 warmup iterations on fixed-clock hardware; report run-to-run variance. Any *quality* delta (TM-score, Foldseek sensitivity, ROC1 where stochastic) uses **5 seeds** with **paired bootstrap 95% CIs** (10,000 resamples) and a **paired Wilcoxon** p-value, following the ICU paper. Correctness deltas reported as max absolute error.

6. Expected Outcomes and Success Criteria

Stated as targets to confirm/refute (no measured values here):

- **Minimum publishable result:** H2 holds — a single Triton source delivers a working **sensitive gapless GPU filter on AMD MI300X** that beats the AMD CPU baseline by $\geq 10\times$, *and* H1's correctness clause holds (bit-exact scores, unchanged ROC1) on NVIDIA. Portability + sensitivity-neutrality alone is a *Bioinformatics*-level contribution.
- **Strong result:** additionally H1 throughput parity ($\geq 0.85\times$ CUDA TCUPS on NVIDIA) and H3 fusion gains ($\geq 50\%$ fewer launches, $\geq 50\%$ of roofline bandwidth on both vendors).
- **Complete result:** additionally H4 — ColabFold TM-score and Foldseek sensitivity statistically unchanged end-to-end, with the hybrid dispatcher delivering best-of-both throughput.
- **Negative/partial results are reportable:** if the Triton SWG kernel underperforms CUDA, the hybrid-dispatch + "when CUDA wins" analysis is itself a useful systems contribution (and does not threaten H2).

7. Threats to Validity and Limitations ("when CUDA wins")

Following the intellectual-honesty precedent of both Triton papers:

- **SWG wavefront** is sequential/data-dependent; Triton may not match CUDASW++4.0 → mitigated by the §4.4 hybrid contingency.
- **Vendor-specific packed math** (`DPX/half2`) gives NVIDIA a structural edge; AMD efficiency (% of peak) may trail even if absolute MI300X throughput is competitive thanks to HBM3 bandwidth. We report % of *per-vendor* peak to avoid an unfair cross-architecture comparison.
- **Atomic contention** in threshold/compaction writes (the same scatter-contention the TensorNet paper flags at scale) — mitigated by per-block local buffering before a single atomic, with an ablation if contention dominates at 8-GPU scale.
- **ROCm/Triton maturity:** autotuning and intrinsic coverage on AMD are younger; we report any kernels where the AMD path needs distinct configs.
- **Benchmark scope:** sensitivity is evaluated on the established SCOP/UniProt set for comparability, not on every domain; downstream accuracy is CASP14/Foldseek, not exhaustive.
- **Correctness ceiling on bf16:** if bf16 ever perturbs scores, we fall back to an exactly-representable accumulation path and report the cost.

8. Reproducibility, Timeline, and Build Sequence

Reproducibility. Open-source Triton kernels + MMseqs2 integration branch + standalone harness; pinned versions (CUDA 12.x, ROCm 6.x, Triton $\geq 3.x$, PyTorch ≥ 2.4); exact commands and dataset snapshots in [02-experimental-protocol.md](#); per-vendor autotune cache published.

Phased build sequence (each phase gated by §4.7 correctness):

1. **Phase 0 — Profiling & baselines:** stand up MMseqs2-GPU CUDA + CPU baselines; profile; produce bottleneck table + roofline. *Gate:* baselines reproduce published sensitivity (ROC1 0.40/0.612/0.669).
2. **Phase 1 — Fused gapless filter (NVIDIA):** implement + autotune + verify bit-exact; hit H1 throughput on NVIDIA. *Gate:* H1 correctness + $\geq 0.85\times$ CUDA TCUPS.
3. **Phase 4a — Port filter to AMD:** compile same source on ROCm; benchmark on MI300X. *Gate:* H2 (runs + beats AMD CPU $\geq 10\times$).
4. **Phase 3 — Fusion increments & launch accounting:** mask/threshold/handoff fusion; ablate. *Gate:* H3 ($\geq 50\%$ launch reduction, bandwidth toward roofline).
5. **Phase 2 — SWG kernel + decision gate:** Triton SWG; if $< 0.7\times$ CUDASW++4.0, activate hybrid dispatcher. *Gate:* end-to-end correctness.
6. **Phase 5 — End-to-end & transfer:** ColabFold CASP14 (TM-score) + Foldseek (SCOPE). *Gate:* H4 (accuracy neutral).
7. **Phase 6 — Economics & write-up:** \$/query, J/query, tail latency; fill results into this plan.

Author roles (to assign): kernel development (Triton); MMseqs2 integration; benchmarking/statistics; biology/downstream validation.

9. References

(Indicative; to be completed in citation style at submission. Bracketed tags correspond to the four core papers provided.)

1. Steinegger, M. & Söding, J. (2017) MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nat. Biotechnol.* 35, 1026–1028. [**core: nbt.3988**]
2. Kallenborn, F., Chacon, A., Hundt, C., Sirelkhatim, H., Didi, K., Cha, S., Dallago, C., Mirdita, M., Schmidt, B. & Steinegger, M. (2025) GPU-accelerated homology search with MMseqs2. *Nat. Methods*. <https://doi.org/10.1038/s41592-025-02819-8> [**core: s41592-025-02819-8**]
3. Singh, M. (2025) Accelerating Molecular Simulations with Triton: Fused GPU Kernels for TensorNet Neural Potentials. *EurIPS Workshop SimBioChem 2025*. [**core: 32_Accelerating_Molecular_Simu**]
4. Anonymous (2025) From 805 ms to 23 ms: Accelerating State-Space Models for Real-Time ICU Monitoring with Fused Triton Kernels. *Under review, ICML*. [**core: 144_From_805_ms_to_23_ms_Accel**]
5. Tillet, P., Kung, H.T. & Cox, D. (2019) Triton: an intermediate language and compiler for tiled neural network computations. *MAPL@PLDI*.
6. Dao, T., Fu, D., Ermon, S., Rudra, A. & Ré, C. (2022) FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*.
7. Schmidt, B., Kallenborn, F., Chacon, A. & Hundt, C. (2024) CUDASW++ 4.0: ultra-fast GPU-based Smith–Waterman protein sequence database search. *BMC Bioinformatics* 25, 342.

8. Buchfink, B., Reuter, K. & Drost, H.-G. (2021) Sensitive protein alignments at tree-of-life scale using DIAMOND. *Nat. Methods* 18, 366–368.
 9. Eddy, S. R. (2011) Accelerated profile HMM searches. *PLoS Comput. Biol.* 7, e1002195.
 10. Altschul, S. F. et al. (1990) Basic local alignment search tool. *J. Mol. Biol.* 215, 403–410; (1997) Gapped BLAST and PSI-BLAST. *Nucleic Acids Res.* 25, 3389–3402.
 11. Smith, T. F. & Waterman, M. S. (1981) Identification of common molecular subsequences. *J. Mol. Biol.* 147, 195–197; Gotoh, O. (1982) *J. Mol. Biol.* 162, 705–708.
 12. van Kempen, M. et al. (2024) Fast and accurate protein structure search with Foldseek. *Nat. Biotechnol.* 42, 243–246.
 13. Mirdita, M. et al. (2022) ColabFold: making protein folding accessible to all. *Nat. Methods* 19, 679–682.
 14. Jumper, J. et al. (2021) Highly accurate protein structure prediction with AlphaFold. *Nature* 596, 583–589.
 15. Steinegger, M. et al. (2019) HH-suite3 for fast remote homology detection and deep protein annotation. *BMC Bioinformatics* 20, 473.
-

End of *01-research-paper-plan.md*. Operational runbook follows in *02-experimental-protocol.md*.